Operating Systems Lab: Source Code and Program Outputs

Attachment / Appendix: CPU Scheduling Implementations and Executions (Labs 8, 9, and 10)

1. Lab 8: First Come, First Served (FCFS) Scheduling

1.1 Source Code (C++)

```cpp
#include <bits/stdc++.h>
using namespace std;

struct Process {
    int id;
    int arrival;
    int execution;
    int start;
    int completion;
    int turnaround;
    int waiting;
};

int main() {
    int n;

    cout << "Enter the number of processes: ";
    cin >> n;

    if (cin.fail() || n <= 0) {
        cout << "Invalid number of processes!\n";
        return 1;
    }

    vector<Process> processes(n);

    // Input process details
    for (int i = 0; i < n; i++) {
        processes[i].id = i + 1;

        cout << "Enter arrival and execution time for process "
             << processes[i].id << ": ";

        cin >> processes[i].arrival >> processes[i].execution;

        if (cin.fail() ||
            processes[i].arrival < 0 ||
            processes[i].execution <= 0) {
            cout << "Invalid input! Arrival time must be "
                 << "non-negative and execution time must "
                 << "be positive.\n";
            return 1;
        }
    }

    // Sort by arrival time; preserve input order for ties
    stable_sort(processes.begin(), processes.end(),
        [](const Process& a, const Process& b) {
            return a.arrival < b.arrival;
        });

    int currentTime = 0;
    double totalWaiting = 0;
    double totalTurnaround = 0;

    // Calculate scheduling times
    for (Process& p : processes) {
        // Handle CPU idle time
        currentTime = max(currentTime, p.arrival);

        p.start = currentTime;
        p.waiting = p.start - p.arrival;

        currentTime += p.execution;

        p.completion = currentTime;
        p.turnaround = p.completion - p.arrival;

        totalWaiting += p.waiting;
        totalTurnaround += p.turnaround;
    }

    // Display results in original process order
    sort(processes.begin(), processes.end(),
        [](const Process& a, const Process& b) {
            return a.id < b.id;
        });

    cout << "\nProcess\tArrival\tExecution\tStart\tCompletion"
         << "\tTurnaround\tWaiting\n";

    for (const Process& p : processes) {
```

```
        cout << "P" << p.id << "\t"
             << p.arrival << "\t"
             << p.execution << "\t\t"
             << p.start << "\t"
             << p.completion << "\t"
             << p.turnaround << "\t\t"
             << p.waiting << '\n';
    }

    // Display average times
    cout << fixed << setprecision(2);

    cout << "\nAverage Waiting Time: "
         << totalWaiting / n << '\n';

    cout << "Average Turnaround Time: "
         << totalTurnaround / n << '\n';

    // Display Gantt chart in FCFS execution order
    stable_sort(processes.begin(), processes.end(),
        [](const Process& a, const Process& b) {
            return a.arrival < b.arrival;
        });

    cout << "\nGantt Chart:\n";

    for (const Process& p : processes) {
        if (p.start > 0 && p.start > p.arrival) {
            // The CPU may have been idle before this process.
            // Idle intervals are displayed below using timestamps.
        }

        cout << "| ";
        if (p.start > 0) {
            cout << "P" << p.id << " ";
        } else {
            cout << "P" << p.id << " ";
        }
    }
    cout << "|\n";

    // Print process start and completion timestamps
    if (!processes.empty()) {
        cout << processes[0].start;

        for (const Process& p : processes) {
            cout << "\t" << p.completion;
        }

        cout << '\n';
    }

    return 0;
}
```

1.2 Execution Output

ratul@firefly: ~/OS-Lab — bash

```
ratul@firefly:~/OS-Lab$ g++ fcfs.cpp -o fcfs
ratul@firefly:~/OS-Lab$ ./fcfs
Enter the number of processes: 4
Enter arrival and execution time for process 1: 0 4
Enter arrival and execution time for process 2: 1 3
Enter arrival and execution time for process 3: 2 1
Enter arrival and execution time for process 4: 3 2
Process  Arrival  Execution  Start  Completion  Turnaround  Waiting
P1       0        4          0      4           4           0
P2       1        3          4      7           6           3
P3       2        1          7      8           6           5
P4       3        2          8      10          7           5
Average Waiting Time: 3.25
Average Turnaround Time: 5.75
Gantt Chart:
| P1 | P2 | P3 | P4 |
0      4      7      8      10
ratul@firefly:~/OS-Lab$ _
```

2. Lab 9: Shortest Job First (SJF Non-Preemptive) Scheduling

2.1 Source Code (C)

```c
#include <stdio.h>

int main() {
    int n;

    printf("Enter the number of processes: ");
    scanf("%d", &n);

    int process[n], arrival[n], burst[n];
    int completion[n], turnaround[n], waiting[n];
    int is_completed[n];

    int i, j, temp, pos;
    int current_time = 0;
    int completed = 0;

    float totalTAT = 0;
    float totalWT = 0;
    float averageTAT, averageWT;

    // Input process details
    for (i = 0; i < n; i++) {
        process[i] = i + 1;

        printf("Enter arrival time of process %d: ", process[i]);
        scanf("%d", &arrival[i]);

        printf("Enter burst time of process %d: ", process[i]);
        scanf("%d", &burst[i]);

        completion[i] = -1;
        is_completed[i] = 0;
    }

    // Sorting processes based on arrival time
    for (i = 0; i < n; i++) {
        pos = i;

        for (j = i + 1; j < n; j++) {
            if (arrival[j] < arrival[pos]) {
                pos = j;
            }
        }

        // Swap arrival time
        temp = arrival[i];
        arrival[i] = arrival[pos];
        arrival[pos] = temp;

        // Swap burst time
        temp = burst[i];
        burst[i] = burst[pos];
        burst[pos] = temp;

        // Swap process ID
        temp = process[i];
        process[i] = process[pos];
        process[pos] = temp;
    }

    // Apply Shortest Job First (Non-Preemptive)
    while (completed != n) {

        int min_burst = 9999;
        int index = -1;

        // Find the shortest available process
        for (i = 0; i < n; i++) {

            if (arrival[i] <= current_time && is_completed[i] == 0) {

                if (burst[i] < min_burst) {
                    min_burst = burst[i];
                    index = i;
                }

                // If burst times are equal, select the one
                // that arrived first
                else if (burst[i] == min_burst &&
                         arrival[i] < arrival[index]) {
                    index = i;
                }
            }
        }

        // If a process is available
        if (index != -1) {
```

```
88
89              completion[index] = current_time + burst[index];
90              turnaround[index] = completion[index] - arrival[index];
91              waiting[index] = turnaround[index] - burst[index];
92
93              // Add to totals
94              totalTAT = totalTAT + turnaround[index];
95              totalWT = totalWT + waiting[index];
96
97              // Update current time
98              current_time = completion[index];
99
100             is_completed[index] = 1;
101             completed++;
102         }
103
104         // If no process has arrived yet
105         else {
106             current_time++;
107         }
108     }
109
110     // Calculate averages
111     averageTAT = totalTAT / n;
112     averageWT = totalWT / n;
113
114     // Printing the final process table
115     printf("\nProcess\tArrival Time\tBurst Time\tCompletion Time\tTurnaround Time\tWaiting Time\n");
116
117     for (i = 0; i < n; i++) {
118         printf("P%d\t%d\t\t%d\t\t%d\t\t%d\t\t%d\n",
119                process[i],
120                arrival[i],
121                burst[i],
122                completion[i],
123                turnaround[i],
124                waiting[i]);
125     }
126
127     // Print total and average values
128     printf("\nTotal Turnaround Time = %.0f", totalTAT);
129     printf("\nTotal Waiting Time    = %.0f", totalWT);
130
131     printf("\nAverage Turnaround Time = %.2f", averageTAT);
132     printf("\nAverage Waiting Time    = %.2f\n", averageWT);
133
134     return 0;
135 }
```

2.2 Execution Output

ratul@firefly: ~/OS-Lab — bash

```
ratul@firefly:~/OS-Lab$ gcc sjf.c -o sjf
ratul@firefly:~/OS-Lab$ ./sjf
Enter the number of processes: 4
Enter arrival time of process 1: 0
Enter burst time of process 1: 7
Enter arrival time of process 2: 2
Enter burst time of process 2: 4
Enter arrival time of process 3: 4
Enter burst time of process 3: 1
Enter arrival time of process 4: 5
Enter burst time of process 4: 4
Process  Arrival Time  Burst Time  Completion Time  Turnaround Time  Waiting Time
P1       0            7           7                7                0
P2       2            4           12               10               6
P3       4            1           8                4                3
P4       5            4           16               11               7
Total Turnaround Time = 32
Total Waiting Time    = 16
Average Turnaround Time = 8.00
Average Waiting Time    = 4.00
ratul@firefly:~/OS-Lab$ _
```

3. Lab 10: Shortest Remaining Job First (SRJF Preemptive) Scheduling

3.1 Source Code (C)

```c
#include <stdio.h>

int main() {
    int n, i, j, temp, pos, current_time = 0, completed = 0;
    float totalTAT = 0, totalWT = 0, averageTAT, averageWT;

    printf("Enter the number of processes: ");
    scanf("%d", &n);

    int process[n], arrival[n], burst[n], remaining[n], completion[n], turnaround[n], waiting[n], is_completed[n];

    // Input process details
    for (i = 0; i < n; i++) {
        process[i] = i + 1;
        printf("Enter arrival time of process %d: ", process[i]);
        scanf("%d", &arrival[i]);
        printf("Enter burst time of process %d: ", process[i]);
        scanf("%d", &burst[i]);
        remaining[i] = burst[i];
        completion[i] = -1;
        is_completed[i] = 0;
    }

    // Sort processes based on arrival time
    for (i = 0; i < n; i++) {
        pos = i;
        for (j = i + 1; j < n; j++) {
            if (arrival[j] < arrival[pos]) pos = j;
        }
        temp = arrival[i]; arrival[i] = arrival[pos]; arrival[pos] = temp;
        temp = burst[i]; burst[i] = burst[pos]; burst[pos] = temp;
        temp = remaining[i]; remaining[i] = remaining[pos]; remaining[pos] = temp;
        temp = process[i]; process[i] = process[pos]; process[pos] = temp;
    }

    // Apply Shortest Remaining Time First (SRTF / Preemptive SJF)
    while (completed != n) {
        int min_remaining = 9999, index = -1;

        // Find process with shortest remaining time among arrived processes
        for (i = 0; i < n; i++) {
            if (arrival[i] <= current_time && is_completed[i] == 0) {
                if (remaining[i] < min_remaining && remaining[i] > 0) {
                    min_remaining = remaining[i];
                    index = i;
                } else if (remaining[i] == min_remaining && arrival[i] < arrival[index]) {
                    index = i;
                }
            }
        }

        if (index != -1) {
            remaining[index]--;
            current_time++;

            if (remaining[index] == 0) {
                completion[index] = current_time;
                turnaround[index] = completion[index] - arrival[index];
                waiting[index] = turnaround[index] - burst[index];
                totalTAT += turnaround[index];
                totalWT += waiting[index];
                is_completed[index] = 1;
                completed++;
            }
        } else {
            current_time++;
        }
    }

    averageTAT = totalTAT / n;
    averageWT = totalWT / n;

    // Printing final process table
    printf("\nProcess\tArrival Time\tBurst Time\tCompletion Time\tTurnaround Time\tWaiting Time\n");
    for (i = 0; i < n; i++) {
        printf("P%d\t%d\t\t%d\t\t%d\t\t%d\t\t%d\n",
               process[i], arrival[i], burst[i], completion[i], turnaround[i], waiting[i]);
    }

    printf("\nTotal Turnaround Time: %.0f", totalTAT);
    printf("\nTotal Waiting Time: %.0f", totalWT);
    printf("\nAverage Turnaround Time: %.2f", averageTAT);
    printf("\nAverage Waiting Time: %.2f\n", averageWT);

    return 0;
}
```

3.2 Execution Output

ratul@firefly: ~/OS-Lab — bash

```
ratul@firefly:~/OS-Lab$ gcc srtf.c -o srtf
ratul@firefly:~/OS-Lab$ ./srtf
Enter the number of processes: 4
Enter arrival time of process 1: 0
Enter burst time of process 1: 7
Enter arrival time of process 2: 2
Enter burst time of process 2: 4
Enter arrival time of process 3: 4
Enter burst time of process 3: 1
Enter arrival time of process 4: 5
Enter burst time of process 4: 4
Process  Arrival Time  Burst Time  Completion Time  Turnaround Time  Waiting Time
P1       0            7           16               16               9
P2       2            4           7                5                1
P3       4            1           5                1                0
P4       5            4           11               6                2
Total Turnaround Time: 28
Total Waiting Time   : 12
Average Turnaround Time = 7.00
Average Waiting Time   = 3.00
ratul@firefly:~/OS-Lab$ _
```